

# WHO AM I?



"Code is my canvas. I don't just build apps—I engineer experiences."

Hey! I'm Muhammad Owais, a passionate Full-Stack Developer and Computer Systems Engineering student at NED University (Class of 2026). With a CGPA of 3.76/4.00, I balance academic rigor with real-world impact.

I specialize in:

- MERN & Django Stacks
- Real-Time Applications (WebSockets, Socket.IO)
- Clean APIs & Modern Frontends (React, Next.js, TailwindCSS)

I'm obsessed with:

- Building efficient backends
- Crafting intuitive UIs
- Solving problems with code

Whether it's creating a social media platform, e-commerce store, or secure real-time chat app, I bring a mix of logic, creativity, and scalability to every line of code.

🔧 Tools I Love:

Django · Django REST Framework · React.js · Next.js · Node.js · Express.js · Python · JavaScript  
PostgreSQL · MongoDB · Sequelize ORM · WebSockets · Socket.IO · Tailwind CSS · Cloudinary

# STAGE1: SINGLE-STORE INVENTORY SYSTEM

“Build a backend service to track product inventory for a single kiryana store.”



## Solutions:

- Simple REST API with SQLite for local storage.
- Tracks stock-ins, sales, and manual removals.

## Key Features:

- “Lightweight database (SQLite).”
- “Basic CRUD operations for products.”
- “Stock updates via API endpoints.”

## Technology Stack for Stage 1

- Node.js: “Runtime environment for executing server-side JavaScript.”
- Express.js: “Web framework for building RESTful APIs and handling HTTP requests.”
- SQLite: “Lightweight, file-based database for local storage of product data.”
- Sequelize: “ORM for defining the Product model and interacting with SQLite.”

Goal: “Efficient and straightforward inventory management for one store.”

# Stage 1 - Design Decisions & Assumptions



## Design Decisions:

- Database: “SQLite chosen for simplicity and local storage—no server setup needed.”
- Model: “Single Product table with name, sku, and quantity for stock tracking.”
- API: “RESTful endpoints over CLI for flexibility and future scalability.”

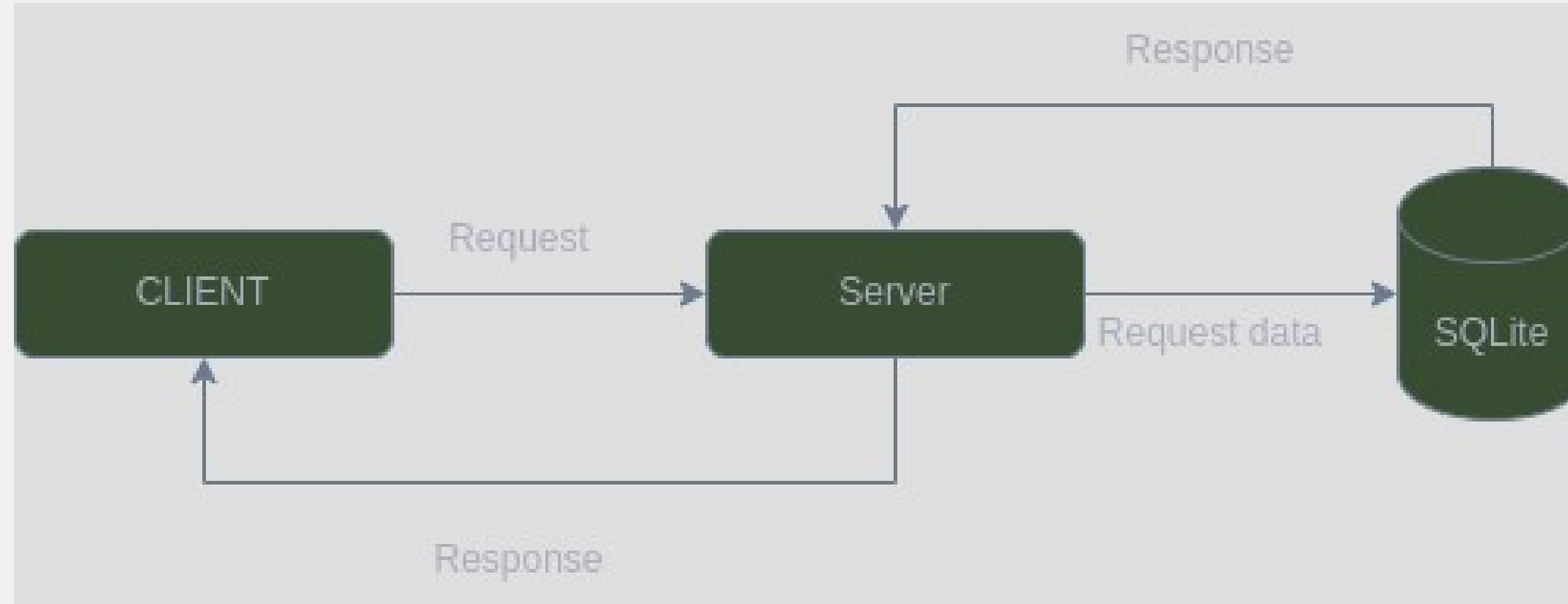
## Assumptions:

- “Single store with multiple products.”
- “No concurrent updates (sequential stock changes).”
- “Manual removals treated as stock-outs.”

Why: “Prioritized ease of use and minimal setup for a small-scale system.”

| id | sku     | name | qunatity |
|----|---------|------|----------|
| 1  | RICE001 | Rice | 50       |
|    |         |      |          |

# Stage 1 - API Design



## API Endpoints:

- POST /api/v1/product/create-productPurpose: Create a product.
  - Request: { "name": "Rice", "sku": "RICE001", "quantity": 10 }
  - Response: { "id": 1, "name": "Rice", "sku": "RICE001", "quantity": 10 }
- GET /api/v1/product/get-all-productsPurpose: Fetch all products.
  - Response: [{ "id": 1, "name": "Rice", "sku": "RICE001", "quantity": 10 }, ...]
- GET /api/v1/product/get-product/:skuPurpose: Fetch product by SKU.
  - Response: { "id": 1, "name": "Rice", "sku": "RICE001", "quantity": 10 }
- PUT /api/v1/product/stock-in/:skuPurpose: Add stock.
  - Request: { "quantity": 5 }
  - Response: { "id": 1, "name": "Rice", "sku": "RICE001", "quantity": 15 }
- PUT /api/v1/product/stock-out/:skuPurpose: Remove stock.
  - Request: { "quantity": 3 }

# Stage2: Multi-Store Inventory System

“Support 500+ stores with a central product catalog, store-specific stock, and audit logs.”



## Solutions:

- REST API with PostgreSQL for relational data.
- Added store-specific products, categories, and inventory logs.

## Key Features:

- Multi-store support with Store and Product models.
- JWT authentication for secure access.
- Inventory logs for auditing stock changes.

## Technology Stack for Stage 2

- Node.js: Server-side runtime.
- Express.js: API framework.
- PostgreSQL: Relational database for multi-store data.
- Sequelize: ORM for database interactions.
- JWT: Authentication via tokens.
- express-rate-limit: API request throttling (100 req/min).

Goal: Scalable and secure inventory management for multiple stores.



# Stage 2 - Design Decisions & Assumptions

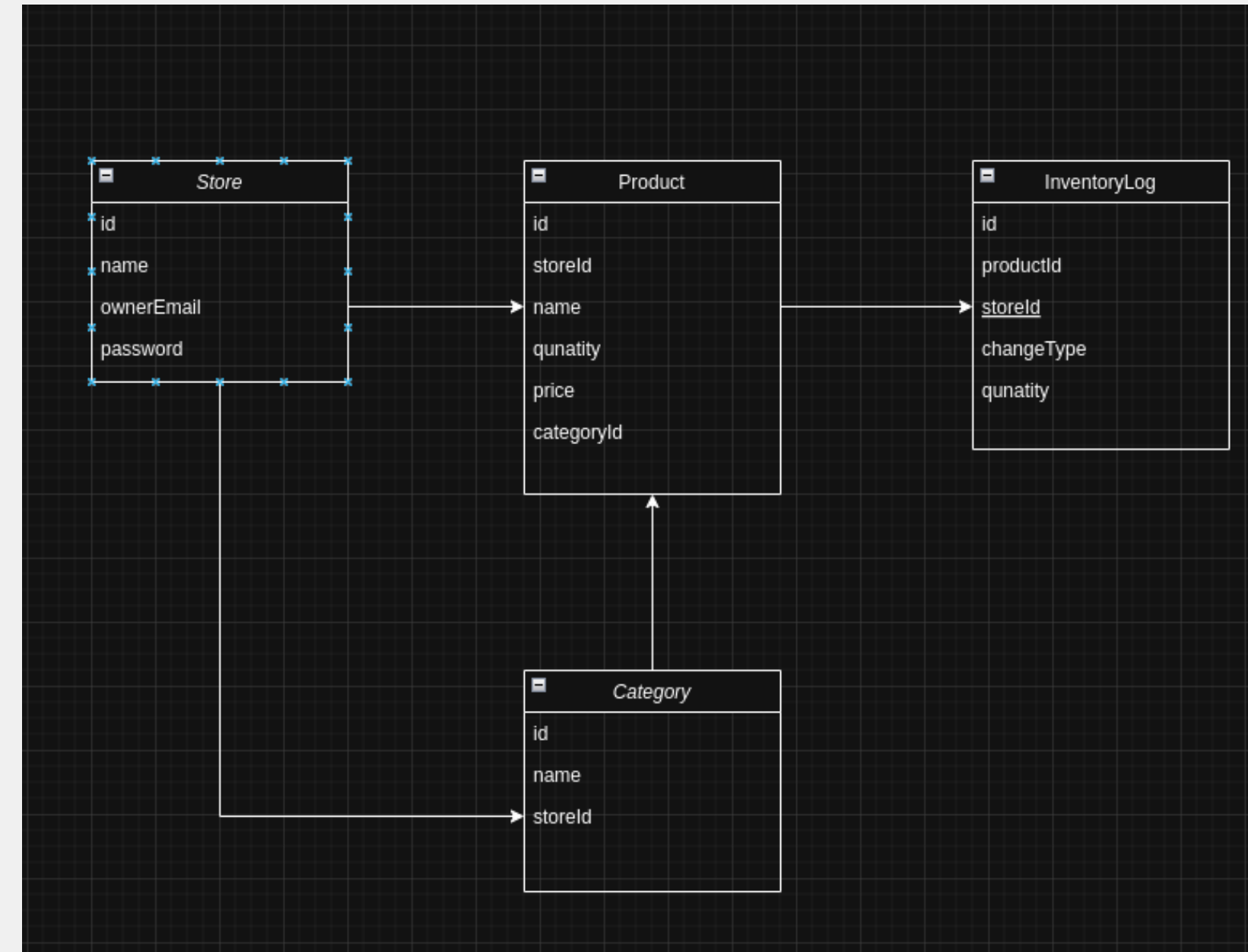


## Design Decisions:

- Database: PostgreSQL for relational integrity and scalability.
- Model: “Store, Category, Product, and InventoryLog for store-specific tracking.”
- Scaling: JWT authentication to restrict access per store.
- Rate Limiting: Added express-rate-limit for API stability.

## Assumptions:

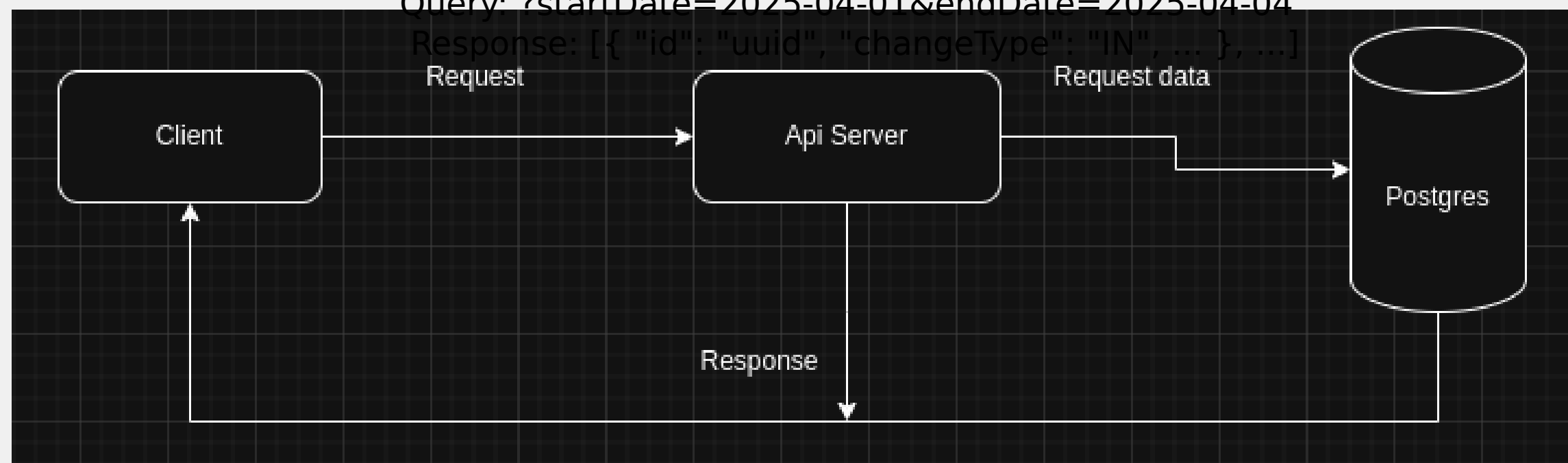
- Each store manages its own products and categories.
- Authenticated users (store owners) access their data only.
- Stock changes are logged for auditing.



# Stage 2 - API Design

## API Endpoints:

- POST /api/v2/store/create-store (24pt)  
Purpose: Create a store (20pt)  
Request: { "name": "Kiryana1", "ownerEmail": "owner@example.com",  
"password": "pass123" }  
Response: { "id": "uuid", "name": "Kiryana1", ... }
- POST /api/v2/product/create-product/:storeId  
Purpose: Create a product for a store (JWT)  
Request: { "name": "Rice", "quantity": 10, "price": 5.99, "categoryId": "uuid" }  
Response: { "id": "uuid", "name": "Rice", ... }
- PUT /api/v2/product/stock-in/:id  
Purpose: Add stock with logging (JWT)  
Request: { "quantity": 5 }  
Response: { "id": "uuid", "name": "Rice", "quantity": 15, ... }
- GET /api/v2/logs/show-logs/:storeId  
Purpose: Fetch logs with date filtering (JWT)  
Query: ?startDate=2025-04-01&endDate=2025-04-04  
Response: [{ "id": "uuid", "changeType": "IN", ... }, ...]



# STAGE3: SCALABLE INVENTORY SYSTEM

“Scale to thousands of stores with real-time stock sync, high concurrency, and auditability.”

“

”



## Solutions:

- Added read/write separation in PostgreSQL for concurrency.
- Introduced Redis for caching and async event-driven updates.

## Key Features:

- Horizontal scalability with master/slave DB setup.
- Redis caching for faster log retrieval.
- Async events via Redis Pub/Sub for near real-time stock updates.

## Technology Stack for Stage 3

- Node.js: Server-side runtime.
- Express.js: API framework.
- PostgreSQL: Relational database for multi-store data.
- Sequelize: ORM for database interactions.
- JWT: Authentication via tokens.
- express-rate-limit: API request throttling (100 req/min).
- Redis: Caching and Pub/Sub for async events.
- moment-timezone: Date handling for log filtering.

Goal: Scalable and secure inventory management for multiple stores.



# Stage 3 - Design Decisions & Assumptions

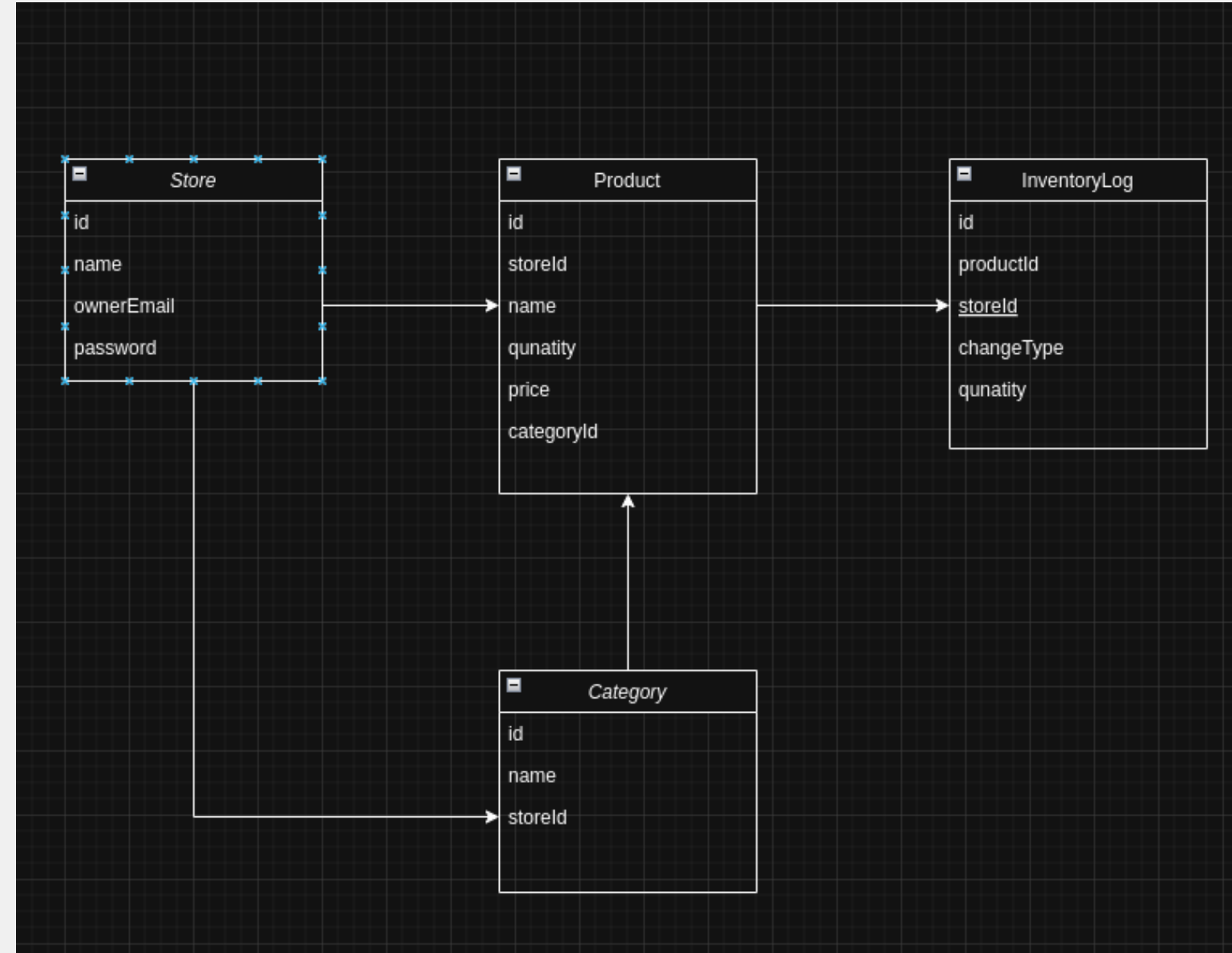


## Design Decisions:

- Database: PostgreSQL with read/write separation (master for writes, slave for reads).
- Caching: Redis for caching inventory logs to reduce DB load.
- Async Events: Redis Pub/Sub for event-driven stock updates.
- Time Handling: moment-timezone for consistent date filtering across regions.

## Assumptions:

- High concurrency handled via DB pooling and caching.
- Stock sync relies on Redis subscribers.
- Cache invalidation ensures data consistency.



# STAGE 3 - API DESIGN

## API Endpoints:

PUT /api/v2/product/stock-in/:id

Purpose: Add stock with logging

Request: { "quantity": 5 }

Response: { "id": "uuid", "name": "Rice", "quantity": 15, ... }

GET /api/v2/logs/show-logs/:storeId

Purpose: Fetch logs with caching and date filtering (JWT)

Query: ?startDate=2025-04-01&endDate=2025-04-04

Response: [{ "id": "uuid", "changeType": "IN", ... }, ...]

DELETE /api/v2/logs/delete-log/:id

Purpose: Delete a log and clear cache (JWT)

Response: { "message": "Log deleted successfully" }

